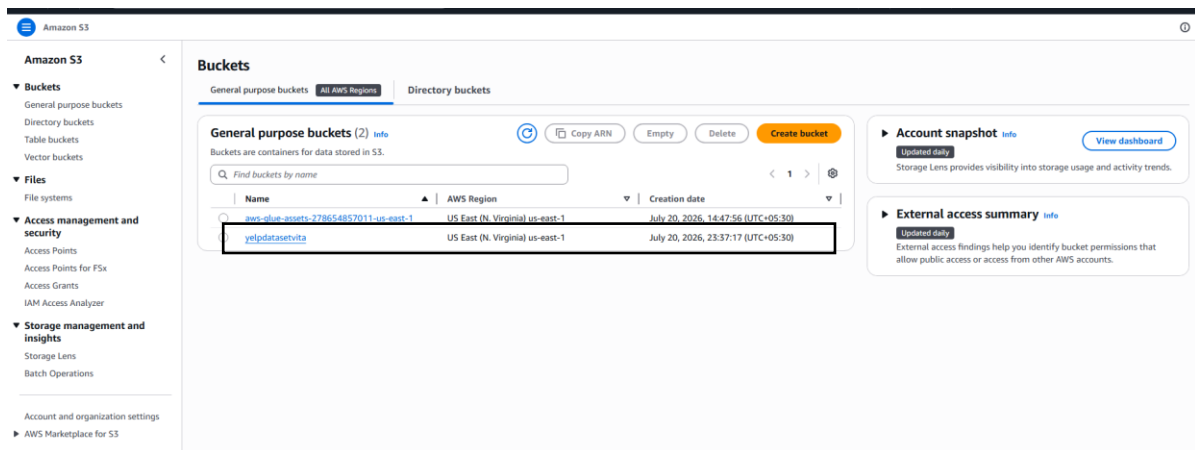
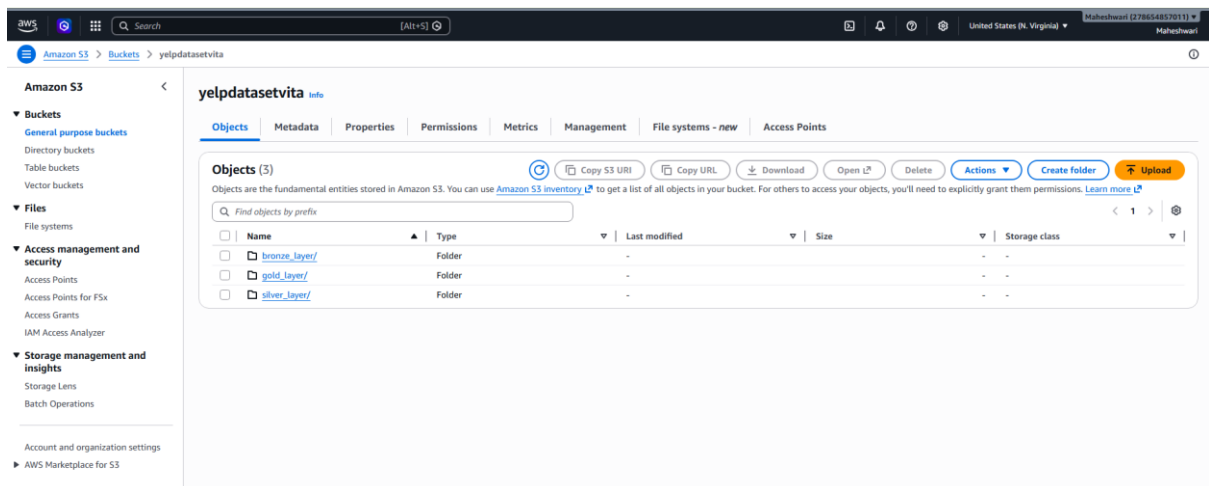


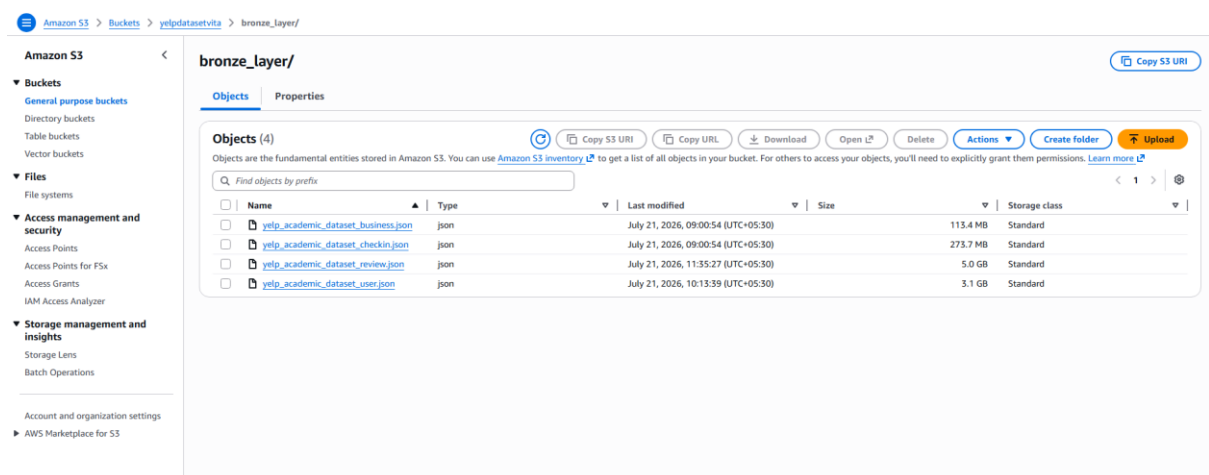
Step 1: Created the bucket named as below:-



Step 2: Created 3 folders inside the bucket for different layers.



Step 3: In Bronze Layer, uploaded 4 JSON files.



Step 4: Created IAM role as:- AWSGlueServiceRole-open, and then added 5 policies to it.

The screenshot shows the AWS IAM console for the role 'AWSGlueServiceRole-open'. The left sidebar contains navigation links for Identity and Access Management (IAM), Access Management, Access reports, and Related consoles. The main content area displays the role's summary, including its creation date (July 21, 2026, 09:35 UTC+05:30), last activity (7 minutes ago), ARN (arn:aws:iam::278654857011:role/service-role/AWSGlueServiceRole-open), and maximum session duration (1 hour). Below the summary, the 'Permissions' tab is active, showing a list of 6 permissions policies. The policies are: AmazonS3FullAccess (AWS managed), AmazonS3OutpostsFullAccess (AWS managed), AmazonS3ObjectLambdaExecutionRolePolicy (AWS managed), AmazonS3TablesFullAccess (AWS managed), AWSGlueServiceRole (AWS managed), and AWSGlueServiceRole-open-EZCRC-s3Policy (Customer managed).

Step 5: Created the database:- yelp_db

The screenshot shows the AWS Glue console 'Databases' page. The left sidebar contains navigation links for AWS Glue, Data Catalog, and Legacy pages. The main content area displays the 'Databases (1)' section, showing a table with one database entry: 'yelp_db'. The table columns are Name, Description, Location URI, Source catalog, and Created on (UTC). The 'yelp_db' entry has a description of '-', a location URI of '-', a source catalog of '278654857011', and a creation date of 'July 20, 2026 at 13:05:00'.

Step 6: Created the Crawler under AWS Glue

- Using the path of Bronze Layer from S3
- Selecting the role and database which we created.

The screenshot shows the AWS Glue console 'Review and update' step for a crawler. The left sidebar contains a progress bar with steps: Step 1: Set crawler properties, Step 2: Choose data sources and classifiers, Step 3: Configure security settings, Step 4: Set output and scheduling, and Step 5: Review and update. The main content area displays the 'Review and update' section, showing the crawler's properties, data sources, security settings, and output and scheduling configuration. The crawler's name is 'yelp-data-crawler'. The data sources are configured with S3 as the type and 's3://yelpdatasetvita/bronze_layer/' as the data source. The security settings are configured with the IAM role 'AWSGlueServiceRole-open'. The output and scheduling configuration shows the database 'yelp_db' and a schedule of 'On demand'.

AWS Glue > Crawlers

AWS Glue

- Getting started
- ETL jobs
 - Visual ETL
 - Notebooks
 - Job run monitoring
- Data Catalog tables
- Data connections
- Workflows (orchestration)
- Zero-ETL integrations [New](#)

▼ **Data Catalog**

- Catalogs
- Databases
- Tables
- Stream schema registries
- Schemas
- Connections
- Crawlers**
- Classifiers
- Catalog settings

► **Data Integration and ETL**

► **Legacy pages**

What's New [🔗](#)
Documentation [🔗](#)
AWS Marketplace

Crawlers

A crawler connects to a data store, progresses through a prioritized list of classifiers to determine the schema for your data, and then creates metadata tables in your data catalog.

Crawlers (1) [Info](#)

View and manage all available crawlers.

Filter crawlers

☐	Name	State	Schedule	Last run	Last run timestamp	Log	Table changes from last r...
☐	yelp-data-crawler	Ready		Succeeded	July 21, 2026 at 07:43:27	View log	-

Last updated (UTC) July 21, 2026 at 07:48:47 [Action](#) [Run](#) [Create crawler](#)

Step 7: Successfully created 4 Glue Jobs for all 4 files

AWS Glue > Jobs

AWS Glue

- Getting started
- ETL jobs
 - Visual ETL
 - Notebooks
 - Job run monitoring
- Data Catalog tables
- Data connections
- Workflows (orchestration)
- Zero-ETL integrations [New](#)

▼ **Data Catalog**

- Catalogs
- Databases
- Tables
- Stream schema registries
- Schemas
- Connections
- Crawlers
- Classifiers
- Catalog settings

► **Data Integration and ETL**

► **Legacy pages**

AWS Glue Studio [Info](#)

Create job [Info](#)

Author in a visual interface focused on data flow. [Visual ETL](#)

Author using an interactive code notebook. [Notebook](#)

Author code with a script editor. [Script editor](#)

► **Example jobs** [Info](#) [Create example job](#)

Your jobs (4) [Info](#)

Filter jobs by property

☐	Job name	Type	Created by	Last modified	AWS Glue version	Action
☐	business-glue-job	Glue ETL	Script	21/07/2026, 13:14:30	5.1	-
☐	review-glue-job	Glue ETL	Script	21/07/2026, 13:07:57	5.1	-
☐	user-glue-job	Glue ETL	Script	21/07/2026, 11:40:33	5.1	-
☐	checkin-glue-job	Glue ETL	Script	21/07/2026, 11:02:37	5.1	-

[Actions](#) [Run job](#)

Step 8: Under business-glue-job, we added the script(in Pyspark) for silver layer. Similarly we added scripts for remaning all 3 files.

business-glue-job Last modified on 21/07/2026, 13:14:30 [Actions](#) [Save](#) [Run](#)

[Script](#) [Job details](#) [Runs](#) [Data quality](#) [Schedules](#) [Version Control](#)

Script [Info](#)

```

1 """
2 AWS Glue ETL Job
3
4 Yelp Business Dataset
5 Bronze (Glue Data Catalog) -> Silver (Parquet + Snappy)
6
7 Source:
8 Database : yelp_db
9 Table : yelp_academic_dataset_business_json
10
11 Destination:
12 s3://yelpdatasetvita/silver_layer/business/
13 """
14
15 import re
16 import sys
17
18 from awsglue.utils import getResolvedOptions
19 from awsglue.context import GlueContext
20 from awsglue.job import Job
21
22 from pyspark.context import SparkContext
23 from pyspark.sql.functions import (
24     col,
25     trim,
26     when,
27     ...

```

Python Ln 1, Col 1 [Errors: 0](#) [Warnings: 0](#)

Step 9: After adding the script, under Job details we added required things or permission

business-glue-job

Script

Job details

Runs

Data quality

Schedules

Version Control

Basic properties

Info

Name

business-glue-job

Description - optional

Descriptions can be up to 2048 characters long.

IAM role

To access other AWS services and resources your Glue ETL job needs an IAM role. Choose a role with the right permissions for your Glue ETL job.

AWSGlueServiceRole-open

Create new role

Edit role

View role details in IAM

Type

The type of ETL job. This is set automatically based on the types of data sources you have selected.

Spark

Glue version

Info

Glue 5.1 - Supports spark 3.5, Scala 2, Python 3

Language

Python 3

Worker type

Set the type of predefined worker that is allowed when a job runs.

G 1X

(4vCPU and 16GB RAM)

Automatically scale the number of workers

☐ AWS Glue will optimize costs and resource usage by dynamically scaling the number of workers up and down throughout the job run. Requires Glue 3.0 or later.

business-glue-job

Script

Job details

Runs

Data quality

Schedules

Version Control

Requested number of workers

The number of workers you want AWS Glue to allocate to this job.

2

Generate job insights

Step 10: After running the glue successfully, in silver file automatically stored the parquet files.

Amazon S3

Buckets

General purpose buckets

Directory buckets

Table buckets

Vector buckets

Files

File systems

Access management and security

Access Points

Access Points for Fsx

Access Grants

IAM Access Analyzer

Storage management and insights

Storage Lens

Batch Operations

Account and organization settings

AWS Marketplace for S3

business/

Objects

Properties

Objects (4)

Find objects by prefix

Copy S3 URI

Copy URL

Download

Open i?

Delete

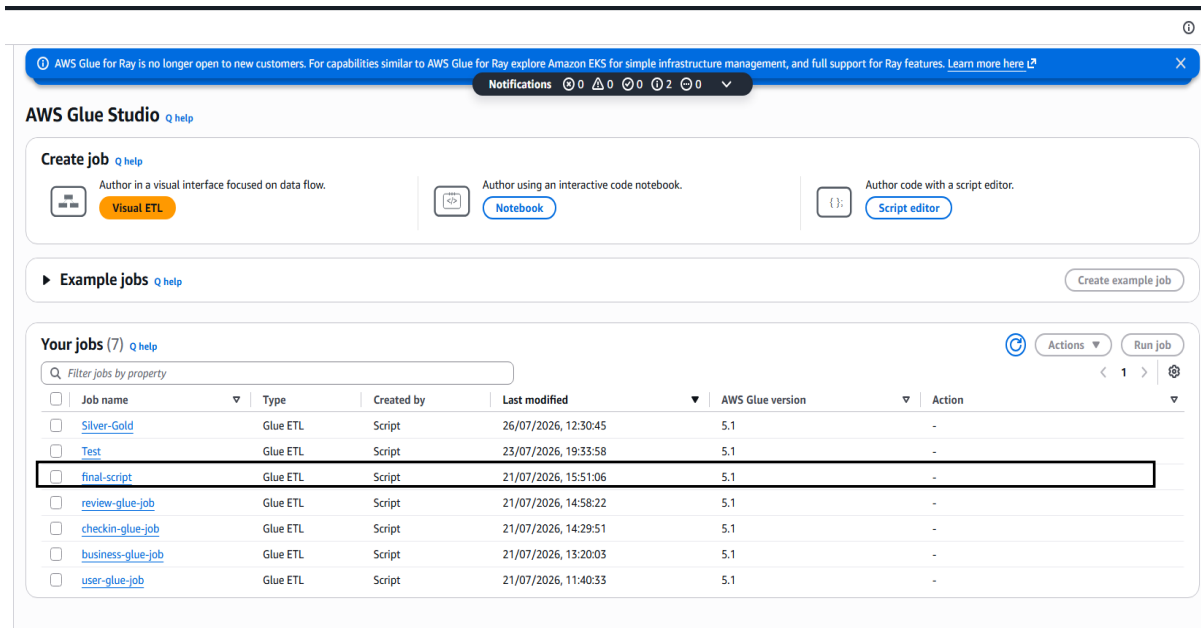
Actions

Create folder

Upload

	Name	Type	Last modified	Size	Storage class
☐	part-00000-976d4405-aebd-4885-9cc5-20a5a0a95954-c000.snappy.parquet	parquet	July 21, 2026, 15:53:36 (UTC+05:30)	4.0 MB	Standard
☐	part-00001-976d4405-aebd-4885-9cc5-20a5a0a95954-c000.snappy.parquet	parquet	July 21, 2026, 15:53:36 (UTC+05:30)	4.1 MB	Standard
☐	part-00002-976d4405-aebd-4885-9cc5-20a5a0a95954-c000.snappy.parquet	parquet	July 21, 2026, 15:53:36 (UTC+05:30)	4.0 MB	Standard
☐	part-00003-976d4405-aebd-4885-9cc5-20a5a0a95954-c000.snappy.parquet	parquet	July 21, 2026, 15:53:36 (UTC+05:30)	4.1 MB	Standard

Step 11: Then one final script we made and run in glue job successfully, which merged all of 4 files.



Hence, successfully created the silver layer.

Expected Questions:-

1. Why were only these IAM policies selected?

Policy	Why it is required
AWSGlueServiceRole	Allows Glue jobs, crawlers, and ETL execution.
AmazonS3FullAccess (or limited S3 access)	Read JSON files from Bronze layer and write Parquet files to Silver layer.
CloudWatchLogsFullAccess	Store Glue job logs for debugging and monitoring.
IAM PassRole (or equivalent permission)	Allows Glue service to assume the IAM role.
AWSGlueConsoleFullAccess (or Glue permissions)	Manage Glue databases, crawlers, and jobs.

2. Why Glue?

"AWS Glue is a serverless ETL service that automatically manages Spark infrastructure, integrates with S3 and the Glue Data Catalog, and efficiently transforms JSON data into Parquet without requiring cluster management."

Scripts:-

1. Business Script

"""

AWS Glue ETL Job

Yelp Business Dataset

Bronze (Glue Data Catalog) -> Silver (Parquet + Snappy)

Source:

Database : yelp_db

Table : yelp_academic_dataset_business_json

Destination:

s3://yelpdatasetvita/silver_layer/business/

"""

import re

import sys

from awsglue.utils import getResolvedOptions

from awsglue.context import GlueContext

from awsglue.job import Job

from pyspark.context import SparkContext

from pyspark.sql.functions import (

col,

trim,

when,

current_timestamp

)

from pyspark.sql.types import StructType, StringType

Glue Boilerplate

-----

args = getResolvedOptions(sys.argv, ["JOB_NAME"])

sc = SparkContext()

glueContext = GlueContext(sc)

spark = glueContext.spark_session

job = Job(glueContext)

job.init(args["JOB_NAME"], args)

-----

Configuration

-----

DATABASE_NAME = "yelp_db"

TABLE_NAME = "yelp_academic_dataset_business_json"

SILVER_PATH = "s3://yelpdatasetvita/silver_layer/business/"

PRIMARY_KEY = "business_id"

-----

Logging

-----

def log(message):

print(f"[BUSINESS_ETL] {message}")

-----

Flatten Struct Columns

-----

```
def flatten_df(df):
```

```
    while True:
```

```
        struct_cols = [
```

```
            field.name
```

```
            for field in df.schema.fields
```

```
            if isinstance(field.dataType, StructType)
```

```
        ]
```

```
        if len(struct_cols) == 0:
```

```
            break
```

```
        for struct_col in struct_cols:
```

```
            expanded_cols = [
```

```
                col(f"{struct_col}.{field.name}").alias(f"{struct_col}_{field.name}")
```

```
                for field in df.schema[struct_col].dataType.fields
```

```
            ]
```

```
        df = df.select(
```

```
            *[
```

```
                c
```

```
                for c in df.columns
```

```
                if c != struct_col
```

```
            ],
```

```
            *expanded_cols
```

```
        )
```

```
    return df
```

```
# -----
```



```
# Standardize Column Names
```

```
# -----
```

```
def standardize_column_name(name):
```

```
    name = name.strip()
```

```
    name = name.replace(" ", "_")
```

```
    name = name.replace("-", "_")
```

```
    name = re.sub(r"^[^a-zA-Z0-9_]", "", name)
```

```
    name = re.sub(r"_+", "_", name)
```

```
    return name.lower()
```

```
# -----
```

```
# Read Bronze Table from Glue Catalog
```

```
# -----
```

```
log("Reading Bronze table from Glue Data Catalog...")
```

```
dynamic_frame = glueContext.create_dynamic_frame.from_catalog(
```

```
    database=DATABASE_NAME,
```

```
    table_name=TABLE_NAME
```

```
)
```

```
business_df = dynamic_frame.toDF()
```

```
log(f"Rows Read : {business_df.count();}")
```

```
log(f"Columns : {len(business_df.columns)}")
```

```
# -----
```

```
# Flatten Nested Struct Columns
```

```
# -----
```

```
business_df = flatten_df(business_df)
```

```
log(f"Columns after flattening : {len(business_df.columns)}")
```

```
# -----
```

```
# Standardize Column Names
```

```
# -----
```

```
business_df = business_df.toDF(
```

```
    *[
```

```
        standardize_column_name(c)
```

```
        for c in business_df.columns
```

```
    ]
```

```
)
```

```
# -----
```

```
# Trim String Columns
```

```
# -----
```

```
string_cols = [
```

```
    field.name
```

```
    for field in business_df.schema.fields
```

```
    if isinstance(field.dataType, StringType)
```

```
]
```

```
for c in string_cols:
```

```
    business_df = business_df.withColumn(c, trim(col(c)))
```

```
# -----
```

```
# Empty String -> NULL
```

```
# -----
```

```
for c in string_cols:
```

```

business_df = business_df.withColumn(
    c,
    when(trim(col(c)) == "", None).otherwise(col(c))
)

# -----
# Validation Report
# -----

null_pk_count = business_df.filter(
    col(PRIMARY_KEY).isNull()
).count()

duplicate_pk_count = (
    business_df
    .groupBy(PRIMARY_KEY)
    .count()
    .filter("count > 1")
    .count()
)

invalid_stars_count = 0
if "stars" in business_df.columns:
    invalid_stars_count = business_df.filter(
        (col("stars") < 1) |
        (col("stars") > 5)
    ).count()

negative_review_count = 0
if "review_count" in business_df.columns:
    negative_review_count = business_df.filter(
        col("review_count") < 0

```

```
).count()
```

```
invalid_latitude = 0
```

```
if "latitude" in business_df.columns:
```

```
    invalid_latitude = business_df.filter(
```

```
        (col("latitude") < -90) |
```

```
        (col("latitude") > 90)
```

```
    ).count()
```

```
invalid_longitude = 0
```

```
if "longitude" in business_df.columns:
```

```
    invalid_longitude = business_df.filter(
```

```
        (col("longitude") < -180) |
```

```
        (col("longitude") > 180)
```

```
    ).count()
```

```
log(f"Null Primary Keys      : {null_pk_count}")
```

```
log(f"Duplicate Primary Keys : {duplicate_pk_count}")
```

```
log(f"Invalid Stars         : {invalid_stars_count}")
```

```
log(f"Negative Review Count : {negative_review_count}")
```

```
log(f"Invalid Latitude      : {invalid_latitude}")
```

```
log(f"Invalid Longitude     : {invalid_longitude}")
```

```
# -----
```

```
# Remove Null Primary Keys
```

```
# -----
```

```
before = business_df.count()
```

```
business_df = business_df.filter(
```

```
    col(PRIMARY_KEY).isNotNull()
```

```
)
```

```
business_df = business_df.filter(  
    trim(col(PRIMARY_KEY)) != ""  
)
```

```
after = business_df.count()
```

```
log(f"Removed {before-after} rows having null/blank primary key")
```

```
# -----  
# Remove Duplicate Records  
# -----
```

```
before = business_df.count()
```

```
business_df = business_df.dropDuplicates(  
    [PRIMARY_KEY]  
)
```

```
after = business_df.count()
```

```
log(f"Removed {before-after} duplicate rows")
```

```
# -----  
# Cast Data Types  
# -----
```

```
cast_map = {
```

```
    "stars": "double",
```

```
    "review_count": "int",
```

```
    "latitude": "double",
```

```

    "longitude": "double",

    "is_open": "int"

}

for column_name, data_type in cast_map.items():

    if column_name in business_df.columns:

        business_df = business_df.withColumn(
            column_name,
            col(column_name).cast(data_type)
        )

# -----
# Add ETL Timestamp
# -----

business_df = business_df.withColumn(
    "etl_processed_timestamp",
    current_timestamp()
)

# -----
# Write Silver Layer
# -----

final_count = business_df.count()

log(f"Writing {final_count:,} rows to Silver Layer...")

```

```
business_df.write \  
    .mode("overwrite") \  
    .option("compression", "snappy") \  
    .parquet(SILVER_PATH)
```

```
log("Silver Layer written successfully.")
```

```
# -----  
# Commit Job  
# -----
```

```
job.commit()
```

```
log("Glue Job Completed Successfully.")
```

2. User Script

```
import sys  
  
from awsglue.utils import getResolvedOptions  
from awsglue.context import GlueContext  
from awsglue.job import Job  
  
from pyspark.context import SparkContext  
  
from pyspark.sql.functions import (  
    col,  
    trim,  
    to_timestamp,  
    current_timestamp  
)  
  
from pyspark.sql.types import StringType  
  
# -----  
# Initialize Glue Job  
# -----  
  
args = getResolvedOptions(sys.argv, ['JOB_NAME'])
```

```

sc = SparkContext()
glueContext = GlueContext(sc)
spark = glueContext.spark_session

job = Job(glueContext)
job.init(args['JOB_NAME'], args)

# -----
# Configuration
# -----

DATABASE_NAME = "yelp_db"
TABLE_NAME = "yelp_academic_dataset_user_json"

OUTPUT_PATH = "s3://yelpdatasetvita/silver_layer/user/"

PRIMARY_KEY = "user_id"

# -----
# Logger
# -----

def log(message):
    print(f"[USER_ETL] {message}")

# -----
# Read Bronze Table from Glue Catalog
# -----

log("Reading Bronze table from Glue Data Catalog...")

dynamic_frame = glueContext.create_dynamic_frame.from_catalog(
    database=DATABASE_NAME,
    table_name=TABLE_NAME
)

df = dynamic_frame.toDF().cache()

log(f"Raw Record Count : {df.count():,}")

# -----
# Remove Duplicate Records
# -----

before = df.count()

df = df.dropDuplicates([PRIMARY_KEY])

after = df.count()

log(f"Removed {before-after} duplicate records")

```



```

# -----
# Remove Missing Primary Key
# -----

before = df.count()

df = df.filter(col(PRIMARY_KEY).isNotNull())
df = df.filter(trim(col(PRIMARY_KEY)) != "")

after = df.count()

log(f"Removed {before-after} null/blank primary keys")

# -----
# Trim Spaces from String Columns
# -----

string_columns = [
    field.name
    for field in df.schema.fields
    if isinstance(field.dataType, StringType)
]

for column_name in string_columns:

    df = df.withColumn(
        column_name,
        trim(col(column_name))
    )

# -----
# Convert Data Types
# -----

if "yelping_since" in df.columns:

    df = df.withColumn(
        "yelping_since",
        to_timestamp(
            col("yelping_since"),
            "yyyy-MM-dd HH:mm:ss"
        )
    )

integer_columns = [

    "review_count",
    "useful",
    "funny",
    "cool",
    "fans",
    "compliment_hot",
    "compliment_more",

```

```

        "compliment_profile",
        "compliment_cute",
        "compliment_list",
        "compliment_note",
        "compliment_plain",
        "compliment_cool",
        "compliment_funny",
        "compliment_writer",
        "compliment_photos"

    ]

    for column_name in integer_columns:

        if column_name in df.columns:

            df = df.withColumn(
                column_name,
                col(column_name).cast("int")
            )

    if "average_stars" in df.columns:

        df = df.withColumn(
            "average_stars",
            col("average_stars").cast("double")
        )

    # -----
    # Fill Missing Values
    # -----

    fill_values = {

        "review_count": 0,
        "useful": 0,
        "funny": 0,
        "cool": 0,
        "fans": 0,
        "average_stars": 0.0

    }

    df = df.fillna(fill_values)

    # -----
    # Add ETL Timestamp
    # -----

    df = df.withColumn(
        "etl_processed_timestamp",
        current_timestamp()
    )

```

```

# -----
# Reduce Output Files
# -----

df = df.coalesce(1)

# -----
# Write Silver Layer
# -----

final_count = df.count()

log(f"Writing {final_count:,} records to Silver Layer...")

(
    df.write
    .mode("overwrite")
    .option("compression", "snappy")
    .parquet(OUTPUT_PATH)
)

log("Silver Layer written successfully.")

# -----
# Commit Job
# -----

job.commit()

log("Glue Job Completed Successfully.")

```

3. Check-in Script

```

"""
AWS Glue ETL Job

Yelp Checkin Dataset
Bronze (Glue Data Catalog) -> Silver (Parquet + Snappy)

Source:
Database : yelp_db
Table   : yelp_academic_dataset_checkin_json

Destination:
s3://yelpdatasetvita/silver_layer/checkin/
"""

import re
import sys

```

```

from awsglue.utils import getResolvedOptions
from awsglue.context import GlueContext
from awsglue.job import Job

from pyspark.context import SparkContext

from pyspark.sql.functions import (
    col,
    trim,
    when,
    current_timestamp
)

from pyspark.sql.types import StringType

# -----
# Initialize Glue Job
# -----

args = getResolvedOptions(sys.argv, ["JOB_NAME"])

sc = SparkContext()
glueContext = GlueContext(sc)
spark = glueContext.spark_session

job = Job(glueContext)
job.init(args["JOB_NAME"], args)

# -----
# Configuration
# -----

DATABASE_NAME = "yelp_db"
TABLE_NAME = "yelp_academic_dataset_checkin_json"

SILVER_PATH = "s3://yelpdatasetvita/silver_layer/checkin/"

PRIMARY_KEY = "business_id"

# -----
# Logger
# -----

def log(message):
    print(f"[CHECKIN_ETL] {message}")

# -----
# Standardize Column Names
# -----

def standardize_column_name(column_name):

```

```

column_name = column_name.strip()
column_name = column_name.replace(" ", "_")
column_name = column_name.replace("-", "_")
column_name = re.sub(r"^[^a-zA-Z0-9_]", "", column_name)
column_name = re.sub(r"_+", "_", column_name)

return column_name.lower()

# -----
# Read Bronze Table from Glue Catalog
# -----

log("Reading Bronze table from Glue Data Catalog...")

dynamic_frame = glueContext.create_dynamic_frame.from_catalog(
    database=DATABASE_NAME,
    table_name=TABLE_NAME
)

checkin_df = dynamic_frame.toDF()

# Cache because count() is used multiple times
checkin_df = checkin_df.cache()

log(f"Rows Read : {checkin_df.count():,}")
log(f"Columns : {len(checkin_df.columns)}")

# -----
# Standardize Column Names
# -----

checkin_df = checkin_df.toDF(
    *[
        standardize_column_name(c)
        for c in checkin_df.columns
    ]
)

# -----
# Trim String Columns
# -----

string_columns = [
    field.name
    for field in checkin_df.schema.fields
    if isinstance(field.dataType, StringType)
]

for column_name in string_columns:

    checkin_df = checkin_df.withColumn(
        column_name,
        trim(col(column_name))
    )

```

```

    )

# -----
# Empty String -> NULL
# -----

for column_name in string_columns:

    checkin_df = checkin_df.withColumn(
        column_name,
        when(
            trim(col(column_name)) == "",
            None
        ).otherwise(col(column_name))
    )

# -----
# Validation Report
# -----

null_pk_count = checkin_df.filter(
    col(PRIMARY_KEY).isNull()
).count()

duplicate_pk_count = (
    checkin_df
    .groupBy(PRIMARY_KEY)
    .count()
    .filter(col("count") > 1)
    .count()
)

log(f"Null Primary Keys    : {null_pk_count}")
log(f"Duplicate Primary Keys : {duplicate_pk_count}")

# -----
# Remove Null / Blank Primary Keys
# -----

rows_before = checkin_df.count()

checkin_df = checkin_df.filter(
    col(PRIMARY_KEY).isNotNull()
)

checkin_df = checkin_df.filter(
    trim(col(PRIMARY_KEY)) != ""
)

rows_after = checkin_df.count()

log(f"Removed {rows_before - rows_after} rows with null/blank business_id")

```

```

# -----
# Remove Duplicate Records
# -----

rows_before = checkin_df.count()

checkin_df = checkin_df.dropDuplicates(
    [PRIMARY_KEY]
)

rows_after = checkin_df.count()

log(f"Removed {rows_before - rows_after} duplicate rows")

# -----
# Add ETL Timestamp
# -----

checkin_df = checkin_df.withColumn(
    "etl_processed_timestamp",
    current_timestamp()
)

# -----
# Reduce Number of Output Files
# -----

checkin_df = checkin_df.coalesce(1)

# -----
# Write Silver Layer
# -----

final_count = checkin_df.count()

log(f"Writing {final_count:,} rows to Silver Layer...")

checkin_df.write \
    .mode("overwrite") \
    .option("compression", "snappy") \
    .parquet(SILVER_PATH)

log("Silver Layer written successfully.")

# -----
# Commit Job
# -----

job.commit()

log("Glue Job Completed Successfully.")

```

4. Review Script

```
import sys

from awsglue.utils import getResolvedOptions
from awsglue.context import GlueContext
from awsglue.job import Job

from pyspark.context import SparkContext

from pyspark.sql.functions import (
    col,
    trim,
    to_timestamp,
    current_timestamp,
    coalesce,
    lit,
    to_date,
    date_format,
    round
)

from pyspark.sql.types import (
    IntegerType,
    DoubleType
)


# -----
# Initialize Glue Job
# -----

args = getResolvedOptions(sys.argv, ['JOB_NAME'])

sc = SparkContext()
glueContext = GlueContext(sc)
spark = glueContext.spark_session

job = Job(glueContext)
job.init(args['JOB_NAME'], args)

print("===== GLUE ETL JOB STARTED =====")

# -----
# Configuration
# -----

DATABASE_NAME = "yelp_db"
TABLE_NAME = "yelp_academic_dataset_review_json"

OUTPUT_PATH = "s3://yelpdatasetvita/silver_layer/review/"
```



```

# -----
# Read Data from Glue Data Catalog
# -----

print("Reading data from Glue Data Catalog...")

dynamic_frame = glueContext.create_dynamic_frame.from_catalog(
    database=DATABASE_NAME,
    table_name=TABLE_NAME
)

df = dynamic_frame.toDF()

initial_count = df.count()

print(f'Initial Record Count : {initial_count}')

# -----
# Remove Duplicate Reviews
# -----

df = df.dropDuplicates(["review_id"])

# -----
# Remove Records with Null Primary Keys
# -----

df = df.filter(
    col("review_id").isNull() &
    col("business_id").isNull() &
    col("user_id").isNull()
)

# -----
# Trim String Columns
# -----

string_columns = [
    "review_id",
    "business_id",
    "user_id",
    "text"
]

for column in string_columns:
    df = df.withColumn(column, trim(col(column)))

# -----
# Convert Data Types
# -----

df = (

```

```

df
# Convert date column to timestamp
.withColumn(
  "date",
  col("date").cast("timestamp")
)

# Extract time into a separate column
.withColumn(
  "time_stamp",
  date_format(col("date"), "HH:mm:ss")
)

# Keep only the date
.withColumn(
  "date",
  to_date(col("date"))
)

# Convert stars to double
.withColumn(
  "stars",
  col("stars").cast("double")
)
)

# Convert useful, funny and cool to integer
# Replace nulls with 0

for c in ["useful", "funny", "cool"]:

    df = df.withColumn(
        c,
        coalesce(
            col(c).cast("int"),
            lit(0)
        )
    )

# -----
# Handle Null Values
# -----

df = df.fillna({
    "text": ""
})

# -----
# Add ETL Timestamp
# -----

df = df.withColumn(
    "etl_load_time",

```

```

        current_timestamp()
    )

# -----
# Calculate Final Count
# -----

final_count = df.count()

print(f'Final Record Count : {final_count}')
print(f'Records Removed : {initial_count - final_count}')

# -----
# Calculate Weighted Score
# -----

df = df.withColumn(
    "weighted_score",
    round(
        (
            0.7 * (col("stars") / 5.0)
            +
            0.3 * (
                (col("useful") + col("funny") + col("cool")) / 30.0
            )
        ) * 10
    ).cast("int")
)

# -----
# Write as Snappy Parquet
# -----

print("Writing Parquet files to S3...")

(
    df.write
        .mode("overwrite")
        .option("compression", "snappy")
        .parquet(OUTPUT_PATH)
)

print("Parquet files successfully written.")

# -----
# Commit Glue Job
# -----

job.commit()

print("===== GLUE ETL JOB COMPLETED =====")

```

5. Final Script

"""

AWS Glue ETL Job — Consolidated Bronze -> Silver

Yelp Academic Dataset

Processes: Business, User, Checkin, Review, Tip

Source : Glue Data Catalog (database "yelp_db")

Target : s3://yelpdatasetvita/silver_layer/<dataset>/ (Parquet + Snappy)

This job runs all five dataset pipelines sequentially inside a single Glue job run. Each dataset keeps its own transformation logic exactly as it existed in its original standalone script — nothing was generalized away or force-fit into a one-size-fits-all cast map. Only genuinely shared plumbing (Glue/Spark init, logging, struct flattening, column-name standardization, trim / blank-to-null) is factored into shared utilities.

NOTE ON THE TIP DATASET:

No source script for Tip was provided. Its configuration below is built from the public Yelp Academic Dataset Tip schema (user_id, business_id, text, date, compliment_count) and mirrors the trim / cast / timestamp / dedupe conventions used across the other four scripts. Replace TIP_ETL's logic if your actual source differs.

"""

```
import re
import sys
```

```
from awsglue.utils import getResolvedOptions
from awsglue.context import GlueContext
from awsglue.job import Job
```

```
from pyspark.context import SparkContext
```

```
from pyspark.sql.functions import (
    col,
    trim,
    when,
    to_timestamp,
    to_date,
    date_format,
    current_timestamp,
    coalesce,
    lit,
    round as spark_round,
)
```

```
from pyspark.sql.types import StructType, StringType
```

```
# =====
```

```

# Glue / Spark Bootstrap (shared across all datasets)
# =====

args = getResolvedOptions(sys.argv, ["JOB_NAME"])

sc = SparkContext()
glueContext = GlueContext(sc)
spark = glueContext.spark_session

job = Job(glueContext)
job.init(args["JOB_NAME"], args)

DATABASE_NAME = "yelp_db"
S3_SILVER_ROOT = "s3://yelpdatasetvita/silver_layer"

# =====
# Shared Utilities
# =====

def log(dataset_tag, message):
    """Uniform structured logging across all dataset pipelines."""
    print(f'[{dataset_tag}_ETL] {message}')

def standardize_column_name(name):
    """
    Lowercases, strips, and replaces spaces/hyphens/illegal characters
    with underscores. Used by Business and Checkin (their original
    scripts both defined this identically).
    """
    name = name.strip()
    name = name.replace(" ", "_")
    name = name.replace("-", "_")
    name = re.sub(r"^[^a-zA-Z0-9_]", "", name)
    name = re.sub(r"_+", "_", name)
    return name.lower()

def flatten_df(df):
    """
    Recursively flattens nested struct columns into
    parent_child-named top-level columns. Used only by Business.
    """
    while True:
        struct_cols = [
            field.name
            for field in df.schema.fields
            if isinstance(field.dataType, StructType)
        ]
        if len(struct_cols) == 0:
            break
        for struct_col in struct_cols:

```

```

        expanded_cols = [
            col(f"{struct_col}.{field.name}").alias(f"{struct_col}_{field.name}")
            for field in df.schema[struct_col].dataType.fields
        ]
        df = df.select(
            *[c for c in df.columns if c != struct_col],
            *expanded_cols,
        )
    return df

```

```

def trim_string_columns(df, columns=None):
    """
    Trims whitespace on the given string columns, or on every
    StringType column if none are specified.
    """
    string_cols = columns or [
        field.name
        for field in df.schema.fields
        if isinstance(field.dataType, StringType)
    ]
    for c in string_cols:
        df = df.withColumn(c, trim(col(c)))
    return df

```

```

def blank_strings_to_null(df, columns):
    """Converts empty/whitespace-only strings to NULL for given columns."""
    for c in columns:
        df = df.withColumn(c, when(trim(col(c)) == "", None).otherwise(col(c)))
    return df

```

```

def read_bronze(tag, table_name):
    log(tag, f"Reading Bronze table '{table_name}' from Glue Data Catalog...")
    dynamic_frame = glueContext.create_dynamic_frame.from_catalog(
        database=DATABASE_NAME,
        table_name=table_name,
    )
    df = dynamic_frame.toDF()
    log(tag, f"Rows Read : {df.count():,}")
    log(tag, f"Columns : {len(df.columns)}")
    return df

```

```

def write_silver(tag, df, output_path, coalesce_to_one=False):
    if coalesce_to_one:
        df = df.coalesce(1)
    final_count = df.count()
    log(tag, f"Writing {final_count:,} rows to Silver Layer at {output_path}")
    df.write \
        .mode("overwrite") \
        .option("compression", "snappy") \

```

```
.parquet(output_path)
log(tag, "Silver Layer written successfully.")
```

```
# =====
# 1. BUSINESS
# =====
```

```
def process_business():
    tag = "BUSINESS"
    table_name = "yelp_academic_dataset_business_json"
    output_path = f"{S3_SILVER_ROOT}/business/"
    primary_key = "business_id"

    df = read_bronze(tag, table_name)

    df = flatten_df(df)
    log(tag, f"Columns after flattening : {len(df.columns)}")

    df = df.toDF(*[standardize_column_name(c) for c in df.columns])

    string_cols = [
        field.name for field in df.schema.fields
        if isinstance(field.dataType, StringType)
    ]
    df = trim_string_columns(df, string_cols)
    df = blank_strings_to_null(df, string_cols)

    # --- Validation report ---
    null_pk_count = df.filter(col(primary_key).isNull()).count()
    duplicate_pk_count = (
        df.groupBy(primary_key).count().filter("count > 1").count()
    )
    invalid_stars_count = (
        df.filter((col("stars") < 1) | (col("stars") > 5)).count()
        if "stars" in df.columns else 0
    )
    negative_review_count = (
        df.filter(col("review_count") < 0).count()
        if "review_count" in df.columns else 0
    )
    invalid_latitude = (
        df.filter((col("latitude") < -90) | (col("latitude") > 90)).count()
        if "latitude" in df.columns else 0
    )
    invalid_longitude = (
        df.filter((col("longitude") < -180) | (col("longitude") > 180)).count()
        if "longitude" in df.columns else 0
    )

    log(tag, f"Null Primary Keys      : {null_pk_count}")
    log(tag, f"Duplicate Primary Keys   : {duplicate_pk_count}")
    log(tag, f"Invalid Stars           : {invalid_stars_count}")
```

```

log(tag, f"Negative Review Count : {negative_review_count}")
log(tag, f"Invalid Latitude : {invalid_latitude}")
log(tag, f"Invalid Longitude : {invalid_longitude}")

# --- Remove null/blank PK ---
before = df.count()
df = df.filter(col(primary_key).isNotNull())
df = df.filter(trim(col(primary_key)) != "")
after = df.count()
log(tag, f"Removed {before - after} rows having null/blank primary key")

# --- Remove duplicates ---
before = df.count()
df = df.dropDuplicates([primary_key])
after = df.count()
log(tag, f"Removed {before - after} duplicate rows")

# --- Cast types ---
cast_map = {
    "stars": "double",
    "review_count": "int",
    "latitude": "double",
    "longitude": "double",
    "is_open": "int",
}
for column_name, data_type in cast_map.items():
    if column_name in df.columns:
        df = df.withColumn(column_name, col(column_name).cast(data_type))

df = df.withColumn("etl_processed_timestamp", current_timestamp())

write_silver(tag, df, output_path, coalesce_to_one=False)

# =====
# 2. USER
# =====

def process_user():
    tag = "USER"
    table_name = "yelp_academic_dataset_user_json"
    output_path = f"{S3_SILVER_ROOT}/user/"
    primary_key = "user_id"

    df = read_bronze(tag, table_name)
    df = df.cache()

    # --- Remove duplicates first (matches original order) ---
    before = df.count()
    df = df.dropDuplicates([primary_key])
    after = df.count()
    log(tag, f"Removed {before - after} duplicate records")

```



```

# --- Remove missing PK ---
before = df.count()
df = df.filter(col(primary_key).isNotNull())
df = df.filter(trim(col(primary_key)) != "")
after = df.count()
log(tag, f"Removed {before - after} null/blank primary keys")

# --- Trim all string columns ---
df = trim_string_columns(df)

# --- Type conversions ---
if "yelping_since" in df.columns:
    df = df.withColumn(
        "yelping_since",
        to_timestamp(col("yelping_since"), "yyyy-MM-dd HH:mm:ss"),
    )

integer_columns = [
    "review_count", "useful", "funny", "cool", "fans",
    "compliment_hot", "compliment_more", "compliment_profile",
    "compliment_cute", "compliment_list", "compliment_note",
    "compliment_plain", "compliment_cool", "compliment_funny",
    "compliment_writer", "compliment_photos",
]
for column_name in integer_columns:
    if column_name in df.columns:
        df = df.withColumn(column_name, col(column_name).cast("int"))

if "average_stars" in df.columns:
    df = df.withColumn("average_stars", col("average_stars").cast("double"))

# --- Fill missing values ---
fill_values = {
    "review_count": 0, "useful": 0, "funny": 0,
    "cool": 0, "fans": 0, "average_stars": 0.0,
}
df = df.fillna(fill_values)

df = df.withColumn("etl_processed_timestamp", current_timestamp())

write_silver(tag, df, output_path, coalesce_to_one=True)

# =====
# 3. CHECKIN
# =====

def process_checkin():
    tag = "CHECKIN"
    table_name = "yelp_academic_dataset_checkin_json"
    output_path = f"{S3_SILVER_ROOT}/checkin/"
    primary_key = "business_id"

```

```

df = read_bronze(tag, table_name)
df = df.cache()

df = df.toDF(*[standardize_column_name(c) for c in df.columns])

string_columns = [
    field.name for field in df.schema.fields
    if isinstance(field.dataType, StringType)
]
df = trim_string_columns(df, string_columns)
df = blank_strings_to_null(df, string_columns)

null_pk_count = df.filter(col(primary_key).isNull()).count()
duplicate_pk_count = (
    df.groupBy(primary_key).count().filter(col("count") > 1).count()
)
log(tag, f"Null Primary Keys : {null_pk_count}")
log(tag, f"Duplicate Primary Keys : {duplicate_pk_count}")

rows_before = df.count()
df = df.filter(col(primary_key).isNotNull())
df = df.filter(trim(col(primary_key)) != "")
rows_after = df.count()
log(tag, f"Removed {rows_before - rows_after} rows with null/blank business_id")

rows_before = df.count()
df = df.dropDuplicates([primary_key])
rows_after = df.count()
log(tag, f"Removed {rows_before - rows_after} duplicate rows")

df = df.withColumn("etl_processed_timestamp", current_timestamp())

write_silver(tag, df, output_path, coalesce_to_one=True)

# =====
# 4. REVIEW
# =====

def process_review():
    tag = "REVIEW"
    table_name = "yelp_academic_dataset_review_json"
    output_path = f"{S3_SILVER_ROOT}/review/"

    df = read_bronze(tag, table_name)
    initial_count = df.count()
    log(tag, f"Initial Record Count : {initial_count}")

    # --- Remove duplicate reviews ---
    df = df.dropDuplicates(["review_id"])

    # --- Remove null primary/foreign keys ---
    df = df.filter(

```

```

        col("review_id").isNotNull()
        & col("business_id").isNotNull()
        & col("user_id").isNotNull()
    )

    # --- Trim specific string columns ---
    string_columns = ["review_id", "business_id", "user_id", "text"]
    df = trim_string_columns(df, string_columns)

    # --- Type conversions ---
    df = (
        df
        .withColumn("date", col("date").cast("timestamp"))
        .withColumn("time_stamp", date_format(col("date"), "HH:mm:ss"))
        .withColumn("date", to_date(col("date")))
        .withColumn("stars", col("stars").cast("double"))
    )

    for c in ["useful", "funny", "cool"]:
        df = df.withColumn(c, coalesce(col(c).cast("int"), lit(0)))

    df = df.fillna({"text": ""})

    df = df.withColumn("etl_load_time", current_timestamp())

    final_count = df.count()
    log(tag, f"Final Record Count : {final_count}")
    log(tag, f"Records Removed : {initial_count - final_count}")

    # --- Weighted score ---
    df = df.withColumn(
        "weighted_score",
        spark_round(
            (
                0.7 * (col("stars") / 5.0)
                + 0.3 * ((col("useful") + col("funny") + col("cool")) / 30.0)
            ) * 10
        ).cast("int"),
    )

    write_silver(tag, df, output_path, coalesce_to_one=False)

# =====
# 5. TIP
# NOTE: No source script was provided for Tip. This follows the
# standard public Yelp Tip schema and mirrors the trim / cast /
# timestamp / dedupe conventions used in the four confirmed
# scripts above. Adjust to your actual source if it differs.
# =====

def process_tip():
    tag = "TIP"

```

```

table_name = "yelp_academic_dataset_tip_json"
output_path = f"{S3_SILVER_ROOT}/tip/"

df = read_bronze(tag, table_name)
initial_count = df.count()
log(tag, f"Initial Record Count : {initial_count}")

# --- Remove records missing required foreign keys ---
df = df.filter(col("business_id").isNotNull() & col("user_id").isNotNull())

# --- Trim string columns ---
string_columns = [
    field.name for field in df.schema.fields
    if isinstance(field.dataType, StringType)
]
df = trim_string_columns(df, string_columns)
df = blank_strings_to_null(df, string_columns)

# --- Remove exact duplicate tips (same user, business, text, date) ---
dedupe_cols = [c for c in ["user_id", "business_id", "text", "date"] if c in df.columns]
before = df.count()
df = df.dropDuplicates(dedupe_cols)
after = df.count()
log(tag, f"Removed {before - after} duplicate tip rows")

# --- Type conversions ---
if "date" in df.columns:
    df = df.withColumn("date", col("date").cast("timestamp"))

if "compliment_count" in df.columns:
    df = df.withColumn(
        "compliment_count",
        coalesce(col("compliment_count").cast("int"), lit(0)),
    )

if "text" in df.columns:
    df = df.fillna({"text": ""})

df = df.withColumn("etl_processed_timestamp", current_timestamp())

final_count = df.count()
log(tag, f"Final Record Count : {final_count}")
log(tag, f"Records Removed : {initial_count - final_count}")

write_silver(tag, df, output_path, coalesce_to_one=True)

# =====
# Orchestration — run every dataset pipeline in this single Glue job
# =====

PIPELINES = [
    ("BUSINESS", process_business),

```

```

    ("USER", process_user),
    ("CHECKIN", process_checkin),
    ("REVIEW", process_review),
    ("TIP", process_tip),
]

failures = []

for name, pipeline_fn in PIPELINES:
    print(f"\n===== STARTING {name} PIPELINE =====")
    try:
        pipeline_fn()
        print(f"===== {name} PIPELINE COMPLETED =====\n")
    except Exception as e:
        print(f"!!!!!!! {name} PIPELINE FAILED: {e} !!!!!!!")
        failures.append(name)

if failures:
    print(f"\nCompleted with failures in: {' '.join(failures)}")
else:
    print("\nAll five dataset pipelines completed successfully.")

# =====
# Commit Job
# =====

job.commit()

print("===== GLUE CONSOLIDATED BRONZE -> SILVER JOB FINISHED =====")

```